AI in Life Sciences

Master Summary

Stefan Eder

Based on Lecture *AI in Life Sciences* SS26

This document is a compact summary of the course. It aims to cover all relevant concepts and results, but long derivations, detailed proofs, and worked examples are often condensed or omitted in favour of clarity and conciseness. Use it as a study aid alongside the lecture notes and slides, not as a replacement for them.

If you spot an error or a gap, please open an issue or pull request at **github.com/stevetgit** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

June 24, 2026

Contents

1

Drug Discovery and the Cheminformatics Toolbox

This first chapter is mostly scene-setting. Before we can let a machine learning model loose on molecules, we need to agree on three things: what problem we are actually trying to solve (*drug discovery*), how we feed a molecule to a computer (*representations*), and what kind of label we are predicting (*bioactivity*). None of this is deep learning yet — but it is the vocabulary the rest of the course is built on, so it is worth getting comfortable with it.

The recurring idea is captured in a single picture: a function $f_{\boldsymbol{\theta}}(\text{molecule}) = \text{activity}$. Everything we do later is just a more elaborate answer to "what is f, and how do we fit its parameters $\boldsymbol{\theta}$ from data?"

1.1 ★ What Drug Discovery Actually Is

At its core, drug discovery is the search for a molecule that **binds to a malfunctioning biological target** and restores its function — while staying *safe* and being *processable by the body*. That last part matters as much as the binding: a molecule that switches off the right protein but is toxic, or that the body cannot absorb or eliminate, is not a drug.

The catch is the price tag. Bringing one drug to market takes roughly **ten years and on the order of a billion dollars**. That is the whole motivation for the course: if computational methods can shave even a slice off that time and cost, the payoff is enormous.

1.1.1 The Development Pipeline

A drug does not go straight from idea to pharmacy. It moves through a pipeline, and each stage filters out candidates:

1. **Target identification** — find the protein (or other biomolecule) whose malfunction drives the disease.
2. **Drug discovery** — find active compounds (*hits* / *leads*), optimise them into drug candidates, and worry about manufacturability (CMC, "chemistry, manufacturing and controls").
3. **Preclinical studies** (in animals) — safety, pharmacokinetics, pharmacodynamics.
4. **Clinical trials** (in humans) — the long, expensive phase.
5. **Approval / launch**.
6. **Post-market surveillance** — watch for rare side effects once millions of people take it.

AI is creeping into every stage (multi-omics analysis and knowledge graphs for target ID, adverse-event monitoring post-market, and so on), but **this course lives almost entirely in stage two**: finding and optimising active small molecules.

1.1.2 A Bit of Pharmacology Vocabulary

Pharmacology is the science of how drugs interact with biological systems. Two words get used constantly and are easy to mix up:

- **Pharmacodynamics (PD)** — what the *drug does to the body* (its mechanism and effect).
- **Pharmacokinetics (PK)** — what the *body does to the drug* (how it is absorbed, distributed, metabolised, excreted).

Add **toxicology** (what harm a chemical can do) and **drug–drug interactions** (how drugs affect each other), and you have the main branches. Drug discovery is really the act of juggling all of these at once.

1.1.3 Two Ways to Find a Drug, and Two Ways to Design One

There is a useful historical split in *how* you go hunting:

- **Phenotypic** (classical / forward pharmacology): screen compounds for a desired *effect* first, identify which lead compounds work, and only *afterwards* figure out which target they hit.
- **Rational** (reverse pharmacology): start from a *known target*, screen compounds for specific interactions with it, then validate the effect in vivo.

Once you start using computers, a parallel split appears in how you *design* candidates — this is **computer-aided drug design (CADD)**:

- **Structure-based drug design (SBDD)** uses the *3D structure* of the target to design molecules that fit snugly into its binding site. Classic tools: molecular docking and simulation. (This is the domain of structural biology — not our focus here.)
- **Ligand-based drug design (LBDD)** uses *known ligands* to design new molecules with similar properties, *without* needing the protein structure. Its toolkit is chemoinformatics: QSAR, pharmacophore modelling, and so on.

Intuition & Mental Model

This whole course is essentially the **ligand-based** story. We will mostly pretend we do *not* have the protein's 3D structure and instead learn patterns purely from molecules and their measured activities. That is exactly the regime where machine learning on molecular representations shines — and where data is plentiful but messy.

1.1.4 ★ The Design–Make–Test–Analyse Loop

Drug discovery is iterative. People talk about the **DMTA cycle**: *design* a molecule, *make* (synthesise) it, *test* it in an assay, *analyse* the result, and feed what you learned into the next design. AI now touches each spoke of the wheel — generative models and matched molecular pairs for design, computer-aided synthesis planning for make, in-silico prediction for test, and sparse-data methods for analyse. Speeding up this loop is the practical goal behind almost everything that follows.

1.2 ★ Small Molecules and How We Represent Them

A drug-like small molecule is mostly a scaffold of **carbon–carbon** and **carbon–hydrogen** bonds, decorated with a few heteroatoms (oxygen, nitrogen, sulfur, halogens). Atoms are held together by *covalent bonds* (shared electrons: single, double, triple, or delocalised "aromatic"), and whole molecules attract each other through weaker *intermolecular forces* (hydrogen bonding, van der Waals, ion–dipole, . . .). These weak forces are not a footnote: they are largely *how a drug grips its target*.

A few structural subtleties matter for activity:

- **3D conformation.** The same molecule can fold into different shapes depending on which bonds can rotate, whether rings are planar, and how it interacts with its environment. Shape drives binding.
- **Isomers.** Molecules with the same formula but different atom arrangements — either constitutional (different connectivity) or stereoisomers (same connectivity, different spatial arrangement).
- **Chirality.** A molecule is *chiral* if it is not identical to its mirror image, like a left and right hand.

Why chirality is not a technicality

The two mirror-image forms (*enantiomers*) of a chiral drug can behave completely differently in the body. The textbook cautionary tale is **thalidomide**: one enantiomer is therapeutic, the other is teratogenic. Two molecules that look almost identical on paper, wildly different effects — a reminder that the *representation* you choose had better be able to tell them apart.

1.2.1 ★ "This Is Not a Molecule"

There is a famous slide riffing on Magritte: a drawing of caffeine captioned *"Ceci n'est pas une molecule."* The point is genuinely important. A SMILES string, a graph, a bit-vector — none of these *is* the molecule. They are all lossy *representations*, and the choice of representation quietly decides what your model can and cannot learn. Picking a representation is the first modelling decision you make, long before you choose an algorithm.

Intuition & Mental Model

There is no single "true" representation. Each one throws away some information and keeps other information handy. A fingerprint makes substructure comparison cheap but forgets 3D shape; a 3D geometry keeps shape but is expensive and conformer-dependent. The art is matching the representation to the question (and to the ML method that consumes it).

1.2.2 ★ The Main Representations

Molecular graphs.. The most natural view: atoms are *labelled vertices*, bonds are *labelled edges*, and the graph is undirected (with some chemical constraints on edges). This is the representation that graph neural networks consume directly (Chapter on advanced methods).

Molecular descriptors.. Hand-engineered numbers summarising a molecule. They are often grouped as constitutional (molecular weight, number of rings, count of hydrogen-bond donors/acceptors), thermodynamic (the octanol–water partition coefficient $\log P$, torsion energy), electronic (topological polar surface area, TPSA), steric (molecular volume), and topological (connectivity indices). People also classify them by "dimension" (0D–4D) depending on how much structure they encode.

Molecular fingerprints.. A fingerprint is a *set of features* extracted from the molecule, then usually *hashed into a fixed-length bit-vector*. Extended-connectivity fingerprints (ECFP / Morgan) are the workhorse: for each atom you look at growing circular neighbourhoods (diameter 0, 2, 4, ...), record the local environment, and hash it into the vector. Hashing is cheap but causes *bit collisions* (two different features landing on the same bit) — a small price for fixed-length vectors you can compare in microseconds.

String representations.. Several text encodings exist; the important ones are:

- **SMILES** (*Simplified Molecular Input Line Entry Specification*) — the dominant one. Atoms are letters (uppercase aliphatic `C`, lowercase aromatic `c`); bonds are implicit single, `=` double, `#` triple; rings use matching digits; branches use parentheses; chirality uses `@`/`@@`. It is compact and ubiquitous — but has *strict syntax*, allows *semantically invalid* strings, and is **non-unique** (one molecule has many valid SMILES).
- **SELFIES** (*Self-referencing embedded strings*) — designed to fix SMILES' fragility: every SELFIES string is a *100% valid* molecule, which is a big deal for generative models.
- **SAFE** — a SMILES-like, fragment-based encoding (fragments separated by dots), handy for fragment-based design.
- **SMARTS** — not for whole molecules but for *substructure patterns* (with wildcards and logical operators); it powers substructure search and feature extraction (e.g. the MACCS keys).
- **InChI / InChIKey** — built for *database lookup*. Because SMILES is non-unique, InChI normalises, canonicalises and serialises a molecule into a *unique* string; the InChIKey is its hashed, fixed-length form.

SMILES is non-unique — mind the gap

Because a single molecule has many valid SMILES, you cannot compare molecules by comparing SMILES strings, and naive train/test splits can accidentally place the "same" molecule on both sides under two different spellings. Canonicalisation (or InChIKeys) is the fix. This non-uniqueness is exactly why InChI exists.

Finally, modern pipelines increasingly use **learned** (continuous) representations produced by deep networks, and **biological** descriptors such as compound-induced transcriptomics, interaction fingerprints, or pharmacophores — representations that try to capture *biological* rather than purely *chemical* similarity. We will come back to that distinction in Chapter 2.

1.3 ★ Biological Activity and Assays

Bioactivity is a compound's ability to interact with a biological system and produce a *measurable response*. We find out about it by running a **bioassay** — a systematic testing procedure that measures the effect of a compound on a biological system and returns a numerical or qualitative outcome. Every bioassay has three parts worth naming:

- the **substance** being tested (e.g. a drug candidate),
- the **target** / biological system (a protein, cell, or organism),
- the **endpoint**, i.e. the measured number (IC_{50}, K_d, % inhibition, ...).

Like any measurement, assays produce *false positives, false negatives and biases* — a theme that will haunt us when we evaluate models. Assays come in flavours: *in vitro* (cell-free or cell-based), *ex vivo* (tissue-based), and *in vivo* (whole organism); they can measure *binding* (affinity to a target) or *function* (an actual biological response); and *high-throughput screening (HTS)* automates testing on a large scale.

1.3.1 Dose–Response Curves and the XC_{50} Family

A compound's effect depends on its *concentration* (in mol/L, usually reported in nanomolar). If you test a substance at many concentrations you trace out a **dose–response curve**, assumed to be sigmoidal. The single number people quote is the concentration producing *half* the maximal effect, generically $\mathbf{XC_{50}}$ — e.g. $\mathbf{IC_{50}}$ is the concentration causing 50% inhibition.

Because these concentrations span many orders of magnitude, we put them on a log scale. The convention (with concentration X in molar) is

$$\mathrm{pX} \;=\; -\log_{10} X.$$

Activity on a log scale

With X in molar, $\mathrm{pX} = -\log_{10} X$. Hence

$$\text{lower } \mathrm{XC_{50}} \iff \text{higher pX} \iff \text{higher activity.}$$

A potent compound has a *small* IC_{50} but a *large* pIC_{50}. Get the direction wrong and your model optimises for exactly the opposite of what you want.

From IC_{50} to pIC_{50}

Suppose an assay reports $IC_{50} = 28\,\mathrm{nM} = 28 \times 10^{-9}\,\mathrm{M}$. Then

$$\mathrm{pIC_{50}} = -\log_{10}\left(2.8 \times 10^{-8}\right) \approx 7.55.$$

A second compound at $IC_{50} = 2.8\,\mu\mathrm{M}$ gives $pIC_{50} \approx 5.55$ — two log units less potent. Modelling pIC_{50} rather than IC_{50} keeps the target variable on a sane, roughly Gaussian scale.

Other affinity measures (K_i, K_d) play the same role. The takeaway: these numbers are the *labels* our QSAR models will try to predict.

1.3.2 ★ Pharmacokinetics: ADME and Drug-Likeness

Good binding is necessary but not sufficient. The compound also has to reach its target and leave the body cleanly. That is summarised by **ADME**:

- **A**bsorption — how it enters the bloodstream,
- **D**istribution — how it spreads to tissues (e.g. crossing the blood–brain barrier),

- **M**etabolism — how it is chemically modified,
- **E**xcretion — how it is cleared.

ADME is genuinely hard to model, so people lean on cheap proxies for "drug-likeness". The classic is **Lipinski's Rule of Five**.

Lipinski's Rule of Five (Ro5)

A compound is more likely to be orally bioavailable if it has *no more than* 5 hydrogen-bond donors, *no more than* 10 hydrogen-bond acceptors, molecular weight < 500, and $\log P < 5$. The **Quantitative Estimate of Drug-likeness (QED)** is a softer, continuous alternative: it scores how closely a molecule's descriptors resemble those of ∼771 approved oral drugs.

1.3.3 ★ Why the Data Is Painful

A theme that will not go away: bioassay data is **limited, imbalanced and noisy**.

- **Expensive labels.** A single high-quality data point can cost up to $10,000 to measure.
- **Small datasets.** Many assays cover just 20–500 molecules; HTS campaigns reach 10k–100k but are noisier.
- **Imbalance.** Almost everything is *inactive*; actives are rare.
- **Noise.** Typical measurement noise is ~ 0.5 log units, and it *grows* when you pool data across different assays.

Set expectations accordingly

Limited + imbalanced + noisy is the default, not the exception. It is why so much of QSAR is about *data-efficient, robust* learning and careful evaluation, rather than chasing the fanciest architecture. Keep this list in mind — it justifies almost every design choice in Chapter 3.

1.4 ★ Where the Data Lives: Databases and Chemical Space

The space of possible drug-like molecules is staggeringly large, and the part we have actually measured is a tiny sliver of it.

Region of chemical space	Approx. size
Estimated drug-like compounds (Lipinski)	10^{33}–10^{60}
Enumerated virtual library (GDB-17)	$\sim 10^{11}$
On-demand commercial library (ZINC-22)	$\sim 10^{10}$
Synthesised / in-stock (PubChem, ZINC20)	$\sim 10^{8}$
Approved drugs (DrugBank)	$\sim 10^{3}$

Table 1.1: Orders of magnitude in chemical space. We have made $\sim 10^8$ molecules and approved $\sim 10^3$ — out of perhaps 10^{33} or more.

For *bioactivity* data specifically there are two camps:

- **Structure-based data** (PDB, PDBBind; > 10k points): you get the binding site and 3D structure, but there are *no negatives* (inactives) and only a small number of ligands.
- **Ligand-based data** (PubChem, ChEMBL; > 100M points): huge, with *labelled actives and inactives* — but no binding conformation.

Since we are doing ligand-based modelling, the second camp is home. The main public databases:

Database	#Compounds	#Bioassay datapoints
PubChem	117 M	294 M
ChEMBL	2.5 M	21 M
Papyrus	1.2 M	60 M
BindingDB	1.3 M	2.9 M
DrugBank	20 K	—

Table 1.2: Public ligand-based bioactivity databases. A "datapoint" is a triple (molecule, assay, value). A lot of additional data sits in private company archives or in non-electronic literature.

The data is mostly holes

Take ChEMBL: ∼2.5M compounds and ∼16k targets, but only ∼21M activity values. If you arrange compounds against assays as a matrix, roughly **0.0005%** of the cells are filled — about a dozen measurements per compound. QSAR modelling is, to a large extent, the problem of doing something useful with an almost-empty matrix.

1.5 ★ From SAR to QSAR (and QSPR)

The empirical bedrock under all of this is the **structure–activity relationship (SAR)**: a molecule's biological activity is determined by its structure, and *similar structures tend to have similar activities*. The *quantitative* version, **QSAR**, claims there is an underlying *mathematical* relationship between structure and bioactivity — precisely our $f_{\theta}(\text{molecule}) = \text{activity}$.

QSAR / QSPR

QSAR (*quantitative structure–activity relationship*) models the *biological* activity of a compound from its structure. **QSPR** (*quantitative structure–property relationship*) does the same for physico-chemical *properties* (solubility, melting point, chemical stability). QSPR data is usually easier and cheaper to obtain, because it does not involve a living system.

QSAR is older than machine learning by a century. A quick lineage: Crum-Brown & Fraser (1868) proposed that physiological action depends on chemical composition; Richet (1893) linked cytotoxicity inversely to water solubility; Meyer and Overton (∼1900) tied narcotic action to oil/water partitioning; and Hansch & Muir (1962) related activity to Hammett constants and hydrophobicity — widely called the *birth of QSAR*.

The *modelling* methods then evolved roughly as:

Era	Dominant methods
< 1990	linear models, simple heuristic relationships
∼1990s	artificial neural networks
∼2000s	SVMs, random forests, gradient boosting
2014–	deep nets, multi-task nets, graph neural nets

Intuition & Mental Model

Notice the through-line: *represent* the molecule as numbers, then *regress or classify* activity from those numbers. The decades just changed *how* we get the numbers and *how flexible* the function f is. With that framing in hand, the next two chapters are simply: how do we *use* such a model to screen a library (Chapter 2), and how do we *build and trust* one in the first place (Chapter 3)?

2

(Ligand-Based) Virtual Screening

We now have molecules, representations, and a notion of activity. The first thing we want to *do* with all that is sift an enormous pile of candidate molecules down to the handful worth testing in the lab. That is **virtual screening (VS)**, and it is the central task of computer-aided drug design.

2.1 ★ The Idea: Screening a Library In Silico

Picture a compound library of more than a million molecules. Testing all of them in assays is expensive or outright infeasible. So we insert a computational filter:

molecule library (>1M) → **virtual screening** → selected molecules → wet-lab assay.

The model evaluates each molecule for the desired property and *prioritises* a small, promising subset for actual experiments. The payoff is concrete: it shrinks the search space, boosts the efficiency of HTS, and cuts cost (HTS alone accounts for ∼15% of pharmaceutical research spending).

Intuition & Mental Model

The mental model is a *funnel*, and the model's job is **ranking**, not perfect prediction. We do not need the exact activity of all million molecules — we need the truly active ones to float to the top of the list so that the limited assay budget is spent where it counts. This "ranking matters" view will shape how we evaluate models in Chapter 3.

2.1.1 Ligand-Based vs. Structure-Based VS

There are two families of methods, mirroring the SBDD/LBDD split from Chapter 1:

- **Ligand-based** VS uses *only ligand information* and leans on SAR. If you know only a *few* actives, you do a **similarity search**; if you have *enough* actives and inactives, you build a predictive **model** (model-based search).
- **Structure-based** VS uses *both* ligand and protein structure. Historically this meant physics-based methods (docking, molecular dynamics, free-energy perturbation); increasingly it means deep-learning docking and co-folding.
- **Hybrid** approaches use both, e.g. proteochemometrics (PCM) models and contrastive methods that bridge several modalities.

The rest of this chapter is **ligand-based**, in two modes: similarity search and model-based search.

2.2 ★ Similarity Search

The premise is the SAR slogan taken literally: if a query molecule is *similar* to a known active, it is probably active too. So we rank a library by similarity to a known active and test the top hits.

The hard part is hiding in the word "similar". Similarity can mean *chemical*, *pharmacological*, or *biological* resemblance, and these do not coincide. Two molecules can look chemically alike yet behave differently, or look chemically different yet hit the same target (as families of MAPK inhibitors do).

Intuition & Mental Model

There is no universally "correct" similarity measure — if there were, drug design would essentially be solved. Every choice of fingerprint and metric encodes a particular guess about *what makes molecules behave alike*. Choosing one is a modelling assumption, not a neutral preprocessing step.

2.2.1 ★ Fingerprint Similarity

The cheapest and most common approach compares binary fingerprints (Morgan/ECFP, RDKit, MACCS). Let a be the number of features (set bits) in molecule A, b the number in B, and c the number they share. Three metrics dominate:

Fingerprint similarity coefficients

$$\text{Tanimoto (Jaccard / IoU):}\quad T(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{c}{a+b-c},$$

$$\text{Dice:}\quad D(A,B) = \frac{2|A \cap B|}{|A| + |B|} = \frac{2c}{a+b},$$

$$\text{Cosine:}\quad C(A,B) = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{c}{\sqrt{a\,b}}.$$

More shared features ⇒ higher similarity. **Tanimoto** is the default in cheminformatics.

Computing all three

Take $a = 3$, $b = 5$, $c = 2$ (molecules with 3 and 5 set bits, sharing 2):

$$T = \frac{2}{3+5-2} = \tfrac{1}{3} \approx 0.33, \qquad D = \frac{2 \cdot 2}{3+5} = 0.5, \qquad C = \frac{2}{\sqrt{15}} \approx 0.52.$$

Same molecules, three different numbers — which is why "how similar are they?" only has an answer once you fix *both* the fingerprint and the metric.

2.2.2 ★ Beyond Fingerprints

Other chemical-similarity notions include the *maximum common substructure*, *graph edit distance*, *Bemis–Murcko scaffolds* (the ring-system backbone of a molecule), and *ECFP kernels*.

A more ambitious idea is **pharmacophore similarity**. A pharmacophore is an *abstract* description of a molecule — typically 3–4 points encoding features like hydrogen-bond donors/acceptors, polarity, hydrophobicity and charge. It is meant to capture *biological* rather than chemical similarity, and is usually handcrafted by experts (there is no agreed-upon definition). One can then define a kernel $k(\cdot, \cdot)$ between the pharmacophore "graphs" of two molecules.

Finally, similarity can be **learned**. Train a neural network whose input is a *chemical* representation and whose output predicts *biological* activity; the hidden layers then form a representation that has been pushed from chemistry towards biology. Comparing molecules with a kernel $k(h(A), h(B))$ on a hidden layer h gives a similarity that reflects biological behaviour better than raw chemical overlap.

Intuition & Mental Model

The arc of this section is a slow migration from *chemical* to *biological* similarity: hard-coded fingerprints → expert pharmacophores → learned embeddings. Each step tries to make "similar" mean "behaves the same in the body" rather than "looks the same on paper".

2.2.3 Running the Search

With a similarity (or kernel) k in hand, the search itself is simple: given a known active, compute $k(\text{active}, m)$ for every molecule m in the library and return the most similar ones. With a million molecules this is just a sparse vector–matrix operation — fast enough to do at scale.

2.3 ★ Model-Based Search

When you have *enough* labelled actives *and* inactives, you can do better than nearest-neighbour similarity: train a **supervised model** that learns its own rules. Concretely, fit a function

$$y = g(\mathbf{x}; \mathbf{w})$$

that maps a molecule's representation $\mathbf{x}$ either to a class (active vs inactive) or to a continuous activity value, then apply it across the library and rank by the predicted score. This is the bridge into QSAR modelling proper, which is the whole of Chapter 3.

2.4 ★ Filtering and Clustering the Hits

A ranked list of predicted actives is not the end. Two practical clean-up steps make the shortlist actually useful.

2.4.1 Filtering Out Problematic Structures

Virtual screening focuses on drug–target interaction, but ADME and toxicity matter just as much for a *viable* candidate. So we drop structures that are likely to cause trouble downstream:

- **Lipinski Rule of Five** — a coarse drug-likeness gate.
- **Brenk filters** — known toxic / reactive substructures.
- **PAINS** (pan-assay interference compounds) — scaffolds notorious for producing *false positives* across many assays.

- **hERG inhibition** — a flag for potential cardiotoxicity via the hERG ion channel.

PAINS: the false-positive trap

PAINS are nasty precisely because they "hit" in lots of assays without being real binders — they interfere with the readout. If you do not filter them, your model happily learns to predict assay artefacts as activity. Filtering is part of honest virtual screening, not an afterthought.

2.4.2 Clustering for Diversity

If your top-ranked hits are all minor variations of one another, testing all of them wastes the assay budget on redundant information. **Clustering** keeps the shortlist *diverse*, maximising the chance of finding genuinely novel chemotypes. The recipe: compute pairwise similarity (from fingerprints or descriptors), cluster, and pick a representative or two from each cluster to span the chemical space.

Two broad clustering styles appear:

- **Distance-based**: k-means, DBSCAN, hierarchical, and *sphere-exclusion* methods (Butina, MaxMin) using a molecular distance (e.g. $1-$Tanimoto).
- **Graph-based**: grouping by Bemis–Murcko scaffold or maximum common substructure.

This idea of similarity-aware grouping comes back with a vengeance in Chapter 3, where the *same* clustering is used not to diversify hits but to build honest train/test splits.

2.4.3 A Note on Tooling

In practice this is all Python. `rdkit` is the backbone — it converts between molecule formats, handles databases, does substructure search via SMARTS, and generates descriptors and fingerprints. Similarity search uses `scipy/numpy/rdkit` for sparse vector operations; classic ML models come from `sklearn` (random forests, SVMs); and neural networks use `PyTorch/TensorFlow/Keras`.

3

QSAR Modeling

Chapter 2 used a model as a black box to rank a library. This chapter opens the box. How do we actually *build* a model f_{θ}(molecule) = activity, and — just as important — how do we *know whether to trust it*? With molecular data, that second question turns out to be where most of the real difficulty lives.

3.1 ★ The QSAR Modelling Pipeline

A QSAR project is a pipeline, and it pays to see the whole assembly line before zooming in:

bioactivity data → molecular representation → data splits → feature selection → model & hyperparameter selection → training → evaluation.

The input is the familiar triple *(assay, molecule, value)* — and, as we keep stressing, that data is limited, noisy, imbalanced and biased. Every box in the pipeline is really a defence against one of those four problems.

3.1.1 ★ Representation and Feature Selection

In principle *any* numerical representation from Chapter 1 can feed a QSAR model; which one works best depends on the ML method paired with it. Feature selection can be applied as usual — with one crucial caveat.

Feature-selection bias

Feature selection must happen *inside* the training loop, using only training data. If you pick features by looking at the whole dataset (test set included), information leaks from test to train and your performance estimate is optimistic. The same discipline applies to scaling, imputation, and any other "learned" preprocessing.

3.1.2 ★ Coping With Limited, Noisy, Imbalanced Data

Recall the pain points from Chapter 1: \$10k labels, datasets of 20–500 molecules, far more inactives than actives, and ∼0.5 log-unit noise. Two pragmatic responses recur throughout QSAR:

- Use **robust, data-efficient** learning methods (the small-data regime rewards strong inductive biases over raw capacity).

- When labels are too noisy for regression, **switch to classification** by thresholding the activity value (active vs inactive). Predicting a coarse label is often more reliable than predicting a noisy number.

3.2 ★ Which Model? Representations Meet Algorithms

The algorithmic menu tracks the historical timeline from Chapter 1: linear models, then SVMs / random forests / gradient boosting, then deep and graph networks. In QSAR the four everyday workhorses are **random forests**, **SVMs**, **gradient boosting**, and **neural networks**.

What matters in practice is that the *representation* and the *method* have to fit each other:

Representation	Methods it pairs well with
Descriptors / fingerprints	linear models, RF, gradient boosting, SVM (standard / Tanimoto kernel), feed-forward NN
SMILES strings	RNN/LSTM, CNN or transformer, SVM (string kernel)
Molecular graph	graph neural networks, SVM (graph / molecule kernel)
Bag of fragments	SVM (Tanimoto kernel), DNN with set pooling

Table 3.1: Common (not exhaustive) representation–method combinations. Empty cells in the lecture's version mean "non-trivial", not "impossible".

Intuition & Mental Model

The honest summary is: **there is no single optimal architecture**. A gradient-boosted model on ECFP fingerprints is a famously hard baseline to beat on small datasets, while graph networks shine when data is plentiful. Picking the representation/method pair is itself a hyperparameter — which is exactly why the next two sections are about *choosing* and *validating* honestly.

3.2.1 Model and Hyperparameter Selection

A lucky break of QSAR is that models are often *cheap* to train (small datasets), so you can afford to try many architectures and hyperparameter settings. The danger is the flip side: try enough configurations and one will look good *by chance*.

Hyperparameter selection bias → use nested CV

Never tune hyperparameters on the same data you report performance on. The clean solution is **nested cross-validation**: an *inner* loop selects hyperparameters (and features), and a separate *outer* loop estimates generalisation. The outer loop never sees the choices the inner loop made on its test folds, so the final number is unbiased.

3.3 ★ The Big QSAR Pitfall: Data Splits

If you remember one thing from this chapter, make it this section. The way you *split* molecular data into train and test sets can quietly inflate your reported performance by a lot.

3.3.1 ★ Compound Series Bias

QSAR datasets are typically built from *series* of closely related molecules — chemists make many small variations on a promising scaffold, and they tend to have similar activity. Now do a *random* train/test split: for almost every test molecule, a near-twin sits in the training set. Prediction becomes trivial, and you wildly **underestimate the generalisation error** — the model looks great in cross-validation and then fails on genuinely new chemistry.

Intuition & Mental Model

Random splits answer the question "can the model interpolate within families it has already seen?" But the question we actually care about in drug discovery is "can it generalise to *new* scaffolds?" Those are very different questions, and a random split silently swaps the hard one for the easy one.

3.3.2 Smarter Splits and Cross-Validation

To estimate the performance we actually care about, make the test set *structurally distant* from the training set:

- **Random split** — the optimistic baseline above.
- **Cluster split** — cluster molecules by structural similarity (with a chemical distance like Tanimoto, via sphere-exclusion, hierarchical, or Butina clustering, or by Bemis–Murcko scaffold), then keep whole clusters on one side of the split. This guarantees a minimum train/test distance and is far more realistic.
- **Temporal / time-series split** — order data by the (first) year the activity was measured and predict the future from the past. This "mimics" how a real drug-discovery project unfolds over time.

The same logic carries into cross-validation: **cluster CV** puts entire clusters of similar molecules into the same fold, and **temporal CV** uses an expanding time window (train on everything up to year t, test on year $t{+}1$).

3.3.3 ★ Multi-Task QSAR and Data Leakage

When one model predicts activity against *many* targets at once (multi-task QSAR), a subtler leak appears: the *same* compound can sit in the training set for one task and in the test set for another. Information bleeds across tasks and you overestimate performance on new compounds. The lecture frames three split strategies and what they trade off:

Split strategy	No leakage	Task balance	Subset dissimilarity
Each task split separately	no	yes	?
Molecules split across all tasks	yes	no	?
… with balancing/stratification	yes	yes	?

The priority order (most to least important) is: *no data leakage* first, then *task balance*, then *subset dissimilarity*. Splitting molecules (not rows) across all tasks, with stratification to keep tasks balanced, is the sweet spot.

3.4 ★ Evaluation and Assessment

Once a model is trained you have to grade it — and with imbalanced, ranking-style problems the choice of metric is not cosmetic. The standard ML metrics all apply, but *which* one you trust depends on what you are using the model for: a ranked list for virtual screening (where **ranking matters**), removing toxic compounds, active learning, and so on.

3.4.1 Regression Metrics

For continuous activity (e.g. pIC_{50}):

- **Accuracy of the value**: MAE (mean absolute error) and RMSE (root mean squared error).
- **Interpretability**: R^2, the coefficient of determination.
- **Ranking**: Spearman's rank correlation ρ — the right metric when you only care about getting the *order* right, which is exactly the VS use case.

3.4.2 ★ Classification Metrics

For active/inactive prediction, start from the confusion matrix (TP, FP, FN, TN). Two families matter:

Threshold ("binary") metrics — computed after you commit to a decision threshold: accuracy, *balanced* accuracy, precision (PPV), recall / sensitivity (TPR), specificity (TNR), F_1, and Matthews' correlation coefficient (MCC).

Ranking ("probabilistic") metrics — computed across all thresholds, so they grade the model's scoring directly:

- **AUC-ROC** — the model's ability to separate positives from negatives across all thresholds.
- **AUC-PR / Average Precision** — the precision/recall trade-off, much more informative than ROC under heavy imbalance.
- **BEDROC** — a variant of ROC that rewards *early recognition* (getting actives near the top of the list).
- **Enrichment Factor (EF)** — how much better than random the true-positive rate is within the top-k predictions.

Imbalance breaks accuracy

Bioactivity classification is overwhelmingly inactive-heavy, so a lazy model that calls *everything* negative can score 99% accuracy while being useless. Always reach for imbalance-robust metrics — precision/recall, F_1, MCC, AUC-PR — and remember BEDROC/EF when what you really want is *good actives at the top of the ranking*.

3.5 ★ Target Prediction vs. Target Identification

So far one model predicts activity against *one* target. Real molecules interact with many proteins, which raises two related but distinct questions.

Drug target prediction asks: *for a molecule, which target(s) does it bind, and how strongly?* Classical QSAR is single-task (affinity to one target). **Multi-task QSAR** solves it with one network that takes a molecule in and emits one *sigmoid* output per target; because related targets share structure, the network can borrow statistical strength across them (and it gracefully handles the missing labels that riddle the activity matrix). Knowledge graphs are an alternative route.

Target identification (a.k.a. *deconvolution* or *target fishing*) asks the complementary question: *given a molecule and several possible targets present at once, which one will it most likely bind?* That is a *multi-class* problem, so the network uses a *softmax* output over targets.

Intuition & Mental Model

The output layer encodes the question. **Sigmoid** units = "is it active at *each* of these targets, independently?" (multi-label / multi-task). **Softmax** units = "which *one* of these competing targets wins?" (multi-class / target identification). Same molecular encoder, different head, different scientific question.

3.6 ★ Drug Synergy

A final, different prediction task: combinations. **Synergy** means the combined effect of two (or more) compounds is *larger than the sum* of their separate effects. Predicting whether a pair is *synergistic* or *antagonistic* is hugely useful — combination therapy is standard for cancer and HIV, while some pairings backfire (grapefruit juice antagonises many drugs and can make them toxic).

How do we even get a label? One common definition is **Loewe synergy**, which decomposes the observed effect of a combination as

$$\underbrace{E}_{\text{observed}} = \underbrace{A}_{\text{additive (expected)}} + \underbrace{S}_{\text{synergy}} .$$

You measure single-agent dose–response curves and the combination's response surface (e.g. a 4×4 grid of concentrations across many cell lines), fit the additive expectation, and read off the synergy S.

Two traps specific to synergy models

(1) Permutation invariance. The prediction for the pair (A, B) must equal the prediction for (B, A) — the model architecture has to be invariant to the order of the molecules.

(2) Data leakage, again. A random split tests performance on combinations you have *already* partly explored. To estimate performance on genuinely *novel* combinations you must split by combination — "leave drug combinations out" (and, more strictly, leave whole drugs or whole cell lines out).

Intuition & Mental Model

Notice the recurring villain of this chapter: **data leakage**. It shows up as random splits over compound series, as compounds shared across multi-task splits, and as already-seen drug combinations. Each fancy split (cluster, temporal, leave-combinations-out) is the same fix applied to a new disguise of the same problem — making sure the test set asks a question the training set has not already answered.

This is where the core cheminformatics arc closes: we can represent molecules, screen libraries, build QSAR models, evaluate them honestly, and extend them to multiple targets and combinations. The *advanced* methods — message-passing graph neural networks, multi-modal and contrastive models, active and few-shot learning, and molecule generation — build directly on this foundation, and are the subject of the chapters that follow.

4

Advanced Methods for QSAR

The QSAR models of Chapter 3 treated molecules as fixed feature vectors (descriptors, fingerprints) and fed them to off-the-shelf learners. This chapter upgrades both ends: representations that are *learned* from the molecular graph itself, and learning settings tailored to the small, biased, multi-modal data that life sciences actually produce — graph neural networks, proteochemometrics and contrastive learning, few-shot and active learning, and multiple-instance learning.

4.1 Molecules as Graphs

A molecule is naturally a **graph** $G = (\mathcal{V}, \mathcal{E})$: atoms are nodes, bonds are edges, and both can carry labels. Graphs are extremely general — images (pixels + adjacency), time series, and sets are all special cases — and their defining nuisance is that *there is no canonical ordering of the nodes*. In machine-learning notation a graph is a pair $(\boldsymbol{X}, \boldsymbol{A})$: a **node-feature matrix** $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ (one feature row per atom) and an **adjacency matrix** $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ encoding which atoms are bonded. Edges (bonds) can carry their own feature matrix too.

Intuition & Mental Model

Keep that "$(\boldsymbol{X}, \boldsymbol{A})$" picture in your head: a molecular graph is **two matrices** — one for the atoms (node features) and one for the bonds (the edges). Typical node features are the *atomic number*, the *hybridization state*, the *formal charge*, and so on; bond type (single/double/aromatic) lives in the edge features. A graph model can and does use *both* — it is not limited to node features.

Because node order is arbitrary, we want two symmetries: node-level representations should be **equivariant** (permute the atoms → the per-atom outputs permute the same way), and graph-level predictions should be **invariant** (permute the atoms → the molecule's predicted property is unchanged). Formally a function g is *invariant* under a group operation u if $g(u(\mathbf{x})) = g(\mathbf{x})$, and *equivariant* if $g(u(\mathbf{x})) = u(g(\mathbf{x}))$. Permutation-invariant pooling (sum/mean/max) is the standard way to get graph-level invariance.

4.2 Graph Neural Networks

4.2.1 The Message-Passing Framework

A **message-passing neural network (MPNN)** learns atom representations by repeatedly letting each atom "talk to" its bonded neighbours. Starting from $\boldsymbol{h}_i^0 = \mathbf{x}_i$ (the atom's input features), each round does three steps:

Message passing (one layer)

$$\begin{aligned} \text{message:} \quad & \boldsymbol{m}_{ij}^{t+1} = \phi\left(\boldsymbol{h}_i^t, \boldsymbol{h}_j^t, \boldsymbol{e}_{ij}\right), \\ \text{aggregate:} \quad & \boldsymbol{m}_i^{t+1} = \textstyle\sum_{j \in N(i)} \boldsymbol{m}_{ij}^{t+1}, \\ \text{update:} \quad & \boldsymbol{h}_i^{t+1} = \psi\left(\boldsymbol{h}_i^t, \boldsymbol{m}_i^{t+1}\right), \end{aligned}$$

where ϕ, ψ are small MLPs and the aggregation must be *permutation-invariant* (it sums over an unordered neighbourhood).

After T rounds, a permutation-invariant **read-out** $\hat{y} = R(\{\boldsymbol{h}_1, \ldots, \boldsymbol{h}_N\})$ pools the atom vectors into one molecule vector for the final prediction — e.g. $\hat{y} = \boldsymbol{\omega}^{\mathsf{T}} \frac{1}{|G|} \sum_{v \in G} \boldsymbol{h}_v$. The choice of aggregation matters: *sum* is the most expressive but propagates signal poorly, *mean*/*max* are smoother but lose information; variance-preserving aggregation is a recent compromise.

Message passing by hand: a 4-atom chain

Intuition first — think of ***gossip****.* Each atom knows a little; every round it tells its bonded neighbours what it currently knows, listens to *all* of them at once, and updates itself. After T rounds each atom has heard from everything within T bonds.

Take the chain $A-B-C-D$ with one-number features h_A^0=1, h_B^0=2, h_C^0=3, h_D^0=4 and neighbourhoods $N(A)$=$\{B\}$, $N(B)$=$\{A, C\}$, $N(C)$=$\{B, D\}$, $N(D)$=$\{C\}$. Use the simplest possible layer — *message* = the neighbour's value, *aggregate* = **sum**, *update* = "add it to myself": $h_i^{t+1} = h_i^t + \sum_{j \in N(i)} h_j^t$. (A real GNN replaces these with small learned MLPs and a nonlinearity, but the bookkeeping is identical — and **what gets aggregated is exactly the neighbours' current vectors**.)

Round 1 (each atom + sum of its neighbours):

$$h_A^1{=}1{+}2{=}3, \quad h_B^1{=}2{+}(1{+}3){=}6, \quad h_C^1{=}3{+}(2{+}4){=}9, \quad h_D^1{=}4{+}3{=}7.$$

Every atom now encodes *itself + its direct neighbours.*

Round 2 (reuse the round-1 numbers):

$$h_A^2{=}3{+}6{=}9, \quad h_B^2{=}6{+}(3{+}9){=}18, \quad h_C^2{=}9{+}(6{+}7){=}22, \quad h_D^2{=}7{+}9{=}16.$$

Key point: h_A^2=9 now depends on C, which is **two bonds away** — information travelled 2 hops in 2 rounds (T layers $\Rightarrow$ a T-hop receptive field).

Read-out: pool the atom values into one molecule vector — e.g. sum = 9+18+22+16=65 — and feed that to an MLP to predict the molecule's property.

A **graph convolutional network (GCN)** is a restricted MPNN that does everything in one matrix step, $\boldsymbol{H}^{l+1} = \sigma(\tilde{\boldsymbol{D}}^{-1/2}\tilde{\boldsymbol{A}}\tilde{\boldsymbol{D}}^{-1/2}\boldsymbol{H}^l\boldsymbol{W}^l)$ with $\tilde{\boldsymbol{A}} = \boldsymbol{A} + \boldsymbol{I}$ (self-loops) — here all neighbours share the same weight. The practical QSAR workhorse is **ChemProp**, a *directed* MPNN (D-MPNN) that passes messages along bonds (each undirected bond becomes two directed edges); for best performance it is combined with classical descriptors. For 3D tasks, **E(n)-equivariant** GNNs (EGNN) additionally track atom coordinates and stay equivariant to rotations/translations.

4.2.2 Where GNNs Struggle

GNNs are not a free lunch, and the lecture is explicit about their failure modes:

- **Limited expressivity** — message passing is at most as powerful as the Weisfeiler–Lehman graph-isomorphism test, so some non-isomorphic graphs are indistinguishable.
- **Oversmoothing** — too many message-passing rounds make all atom representations collapse to the same value.
- **Oversquashing** — the influence of distant atoms decays exponentially with path length, so **long-range interactions** between atoms are hard to capture.
- **Under-reaching** — with too few rounds, information simply doesn't travel far enough across a large molecule.

GNN vs. Random Forest — a realistic comparison

GNNs are not "always better". On the small, feature-rich datasets typical of QSAR, a **random forest** often *overfits less* and is *more interpretable* (you can read off feature importances) than a GNN, while GNNs are compute-hungry, struggle with very large/complex graphs and long-range effects, and are *not* less sensitive to noisy data. The honest verdict from Chapter 3 still holds: no single architecture wins everywhere.

4.3 Using Target Information: Proteochemometrics and Multimodal Models

Plain QSAR has **no target awareness**: $f_{\boldsymbol{\theta}}(\text{molecule})$ knows nothing about *which* protein it is predicting activity for, so it cannot generalise to a new target. **Proteochemometrics (PCM)** fixes this by giving the model *both* the molecule and a representation of the target: $f_{\boldsymbol{\theta}}(\text{molecule}, \text{target})$. The target can be encoded by handcrafted amino-acid descriptors, learned from the sequence, or taken from a pretrained protein language model (e.g. ESM). Pooling data across targets gives *more* training data and lets the model learn cross-target correlations, at the price of limited applicability to truly novel targets. PCM is one instance of a **multimodal model** — an architecture that integrates several modalities (molecule, protein, image, text) and fuses them early, jointly, or late to exploit their complementary information.

4.3.1 Contrastive Learning

A particularly powerful multimodal recipe is **contrastive learning**, a *self-supervised* technique that learns by pulling matching pairs together and pushing mismatched pairs apart in a **shared latent space**. CLIP is the canonical example (images $\leftrightarrow$ captions); in drug discovery the pairs are *(molecule, X)* where X is a binding pocket, a protein sequence, a cell image, or an assay description (CLAMP, DrugCLIP, CLOOME, SPRINT). Because the data can be huge (>100M assay pairs), the learned embeddings support *zero-shot* generalisation to new tasks.

Contrastive learning — the exact wording

Contrastive learning encodes multiple modalities into a **shared latent space** and is **self-supervised** (not unsupervised). It *aligns* representations of similar/dissimilar pairs; it does not "concentrate representations to make predictions" — that description is the distractor.

4.4 Learning from Little Data

4.4.1 Few-Shot and Zero-Shot Learning

A brand-new target may come with < 10 measured actives — far too few for a standard QSAR/PCM model. **Few-shot learning** explicitly adapts to a small labelled *support set*, while **zero-shot** makes predictions with *no* task-specific data at all. The main routes are *pretraining + fine-tuning / transfer learning*, *in-context / retrieval-based* methods that condition on example compounds without weight updates (e.g. MHN-FS, a Modern Hopfield Network), and *meta-learning* (e.g. MAML) which trains across many tasks to learn rapid adaptation.

4.4.2 Active Learning

When labels are expensive, don't label randomly — let the model pick. **Active learning** iteratively trains a model, has it select the *most informative* molecules from the screening library, sends those for experimental labelling, and retrains. "Informative" usually means *high prediction uncertainty* (near the decision boundary or in under-explored chemical space), often combined with *diversity filters* to avoid picking near-identical compounds. Uncertainty itself is estimated by ensembling, Monte-Carlo dropout, or evidential learning, and should be *calibrated* (ECE, Brier score).

4.5 Multiple Instance Learning

Sometimes the label belongs not to a single molecule but to a *set*. **Multiple instance learning (MIL)** trains on **bags** $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_M\}$ that carry a *single* label y — the individual instances are *not* labelled. The classic chemistry motivation (Dietterich, 1997) is a molecule with many 3D *conformations*: the molecule is active if *some* conformation binds, but we don't know which one.

What MIL requires (and what it doesn't)

A MIL model must be **invariant to permutations** of the instances, $\hat{y} = g(\{\mathbf{x}_1, \ldots, \mathbf{x}_M\}) = g(\pi(\{\mathbf{x}_1, \ldots, \mathbf{x}_M\}))$, and must **handle variable bag sizes**. The bag has *one* label (not one per instance), and the size is *not* fixed. The fraction of instances that actually indicate the bag's class is the *witness rate*.

Architecturally, MIL uses "Deep-Sets"-style models: encode each instance, then apply a *permutation-invariant pooling* (mean/max/sum/attention) and an output layer. *Bag-level* scoring (pool representations, then predict) usually beats *instance-level* scoring (predict per instance, then pool). Attention-based MIL learns which instances matter. Finally, **large language models** are an emerging — if still unreliable — tool for molecular reasoning, strong on chemistry knowledge but weak at understanding raw chemical structure.

5

Generative Models for Molecules

So far every model has been *discriminative*: given a molecule, predict a number. Generative models run the arrow the other way — *produce* new molecules with desired properties. This is **de novo drug design**: where virtual screening is $f_{\boldsymbol{\theta}}(\text{molecule}) = \text{property}$, generation is $\text{molecule} = g_{\boldsymbol{\gamma}}(\text{property})$. The motivation is the sheer size of chemical space: $\sim 10^8$ molecules have ever been synthesised, but there are perhaps $\sim 10^{60}$ drug-like ones — screening can only ever look at a sliver, so why not *design* directly?

5.1 ★ How to Generate a Molecule

Generators differ in the *representation* they emit:

- **String-based** — generate a SMILES or SELFIES string token by token (autoregressive RNN/LSTM/transformer).
- **Graph-based** — build the molecular graph directly, atom by atom (autoregressive) or all at once (matrix-based: sample an adjacency tensor and node matrix).
- **Rule-based** — apply transformation rules to starting compounds.
- **3D** — generate atom coordinates (diffusion models, equivariant).

The most common starting point is **autoregressive SMILES generation**: encode the string in one-hot (chemically meaningful tokens), then predict the next token with $\hat{P}_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_t \hat{P}_{\boldsymbol{\theta}}(x_t \mid x_{t-1}, \ldots, x_1)$. An LSTM learns the awkward SMILES syntax surprisingly well.

Generative models — the facts the exam loves

Autoregressive models *can* generate SMILES; an LSTM reaches up to **∼98% valid** strings, and with **SELFIES** validity is **100%**. So the claim that SMILES generators produce "<10% valid" is false. Generative models explore *novel* chemical space (not just the *known* synthesised set), and **GANs absolutely can** generate molecules (alongside VAEs, normalizing flows, and diffusion models).

5.2 ★ Three Training Paradigms

Distribution learning. Learn to *mimic* the property distribution of a large library of real molecules (PubChem/ChEMBL) and reproduce its "syntax" to emit realistic molecules. Self-supervised; used to expand VS libraries and as a *starting point* for goal-directed design. Models: autoregressive, VAE, GAN, normalizing flow, diffusion.

Goal-directed learning. *Optimise* molecules for a **specific objective** (e.g. bioactivity) using a **scoring function** and an iterative loop.

Conditional generation. Generate molecules that *explicitly satisfy* an input condition (property value, scaffold, 3D pocket) by learning a shared latent space of molecules and conditions.

Distribution learning vs. goal-directed

A subtle exam distinction: *distribution learning* aims to create molecules **similar to the training set**; *goal-directed* generation aims to create molecules with **specific desired properties** and can be optimised with **reinforcement learning**. "Replicating existing structures" is *not* the point of goal-directed design.

5.2.1 ★ Goal-Directed Optimisation in Practice

The optimisation loop can be hill-climbing (iterative distribution learning), *genetic algorithms*, *reinforcement learning* (define an agent, generate, score, reward, retrain — e.g. REINVENT), or continuous optimisation in a VAE latent space. Real drug design is **multi-objective** (maximise efficacy, selectivity, QED, synthesizability; minimise toxicity, lipophilicity), so there is no single best molecule — trade-offs are handled by *score aggregation* (weighted sums) or *Pareto ranking* (keep non-dominated solutions). A nasty failure mode is **mode collapse**: the generator keeps proposing the same or very similar molecules, whereas we want *diverse* candidates to maximise downstream success.

The natural way to *drive* an autoregressive generator toward a goal is to read it as a **reinforcement-learning** problem — building a molecule one step at a time *is* a sequential decision process. The *state* s is the partial molecule (a SMILES prefix or partial graph); an *action* a is the next construction step (next token, atom, bond, fragment, or reaction); the *policy* $\pi(a \mid s)$ *is* the generative model (a distribution over next actions); an *episode* τ is one finished molecule; and the *reward* $R(\tau)$ is the score of that final molecule (activity, QED, SA, similarity, ...). Autoregressive (causal, next-token) models couple to RL especially cleanly because the policy already factorises over construction steps — but the reward structure makes the problem nasty.

★ **Why molecular RL is hard**

- **Delayed reward** — the score is known only *after* a full molecule has been generated, not step by step.
- **Sparse / weak signal** — early in training most molecules are invalid, inactive, or low-scoring.
- **Proxy objective & reward hacking** — the reward is a QSAR, docking, or heuristic score (*not* experimental success), so the generator can exploit *artifacts* of the scoring function.
- **Diversity is not automatic** — unless explicitly rewarded, RL does not try to cover multiple chemical solutions.
- **Distribution shift** — aggressive optimisation drifts the agent away from the realistic chemistry the prior had learned.

5.2.2 ★ Conditional Generation

Conditional generators (conditional VAE/RNN/transformer/diffusion) give *direct control* over properties without a scoring function and enable fast one-pass generation, but they need *paired* data (molecules with known properties), may only *approximately* satisfy the target, generalise poorly to rare conditions, and risk *property entanglement*.

5.3 ★ Latent-Variable Models and Latent-Space Optimisation

Autoregressive models build a molecule step by step; the *latent-variable* alternative is to first learn a **continuous space** of molecules (typically with a VAE) and then generate by sampling a latent vector $\mathbf{z}$ and *decoding* it. New molecules come from sampling $\mathbf{z}$; goal-directed design becomes *moving* $\mathbf{z}$ toward desired properties — optimisation in a smooth space instead of surgery on discrete graphs.

The decoder depends on the representation. A latent $\mathbf{z}$ is turned back into a molecule by a *sequential SMILES decoder* (token sequence), a *graph decoder* (atoms + bonds), or a *fragment decoder* (motifs / scaffold pieces). Graph decoding in particular comes in three flavours:

One-shot — predict all atom types and the bond-adjacency tensor at once. Conceptually simple, but hard to enforce valence and connectivity.

Sequential — $\mathbf{z}$ + partial graph → next atom/bond action; builds step by step and can *restrict actions to chemically valid choices* (like graph autoregression, but conditioned on $\mathbf{z}$).

Fragment-level — $\mathbf{z}$ → chemically meaningful fragments that are assembled into the graph; often improves validity and connects naturally to scaffold/linker tasks, but is limited by the fragment vocabulary.

Why optimise in latent space? Molecules are *discrete*, but $\mathbf{z}$ is *continuous*. We train (or reuse) a property model $f(\mathbf{z})$, search $\mathbf{z}^* = \arg\max_{\mathbf{z}} f(\mathbf{z})$, then decode $x^* \sim p(x \mid \mathbf{z}^*)$. Continuity unlocks tools that discrete graphs do not offer: *gradient ascent* (if f is differentiable), *Bayesian optimisation* (when evaluations are expensive), *evolutionary search* (mutate/select latent vectors), and *interpolation* between known actives. The learned prior keeps the search near realistic chemistry, similar molecules tend to sit close together when the space is well learned, and sampling many $\mathbf{z}$ naturally yields multiple candidates — at the cost of only *approximate* control over the decoded structure.

5.4 ★ Diffusion Models and 3D Generation

Diffusion models generate by denoising: start from a noisy representation and learn to *reverse* a gradual noising process, producing a molecule through *iterative* refinement rather than one-shot decoding. This iterative view is especially natural for *continuous* representations such as 3D coordinates.

What gets denoised matters. Two regimes:

2D graph diffusion — generates molecular *identity* (atoms + bonds); noise is applied to *discrete* atom/bond types and denoising predicts the clean graph. The hard part is *chemical validity*: valence, connectivity, aromaticity.

3D diffusion — generates identity *and/or* geometry; noise is applied to *continuous* coordinates (sometimes also atom types). The hard part is *realistic geometry*: stable structures and correct symmetries, which demands rotation/translation-aware models.

★ 3D generation needs equivariance

Molecular coordinates have **no fixed global orientation**, so a 3D generator must not depend on an arbitrary coordinate frame. If you *rotate or translate* the input structure, the prediction should rotate/translate *consistently* — this property is **equivariance**, and *equivariant networks* build it in by construction. Ignore it and the model wastes capacity memorising orientations and tends to produce unstable, unrealistic geometries.

Conditional 3D diffusion fixes part of the molecule and denoises the rest:

- **Conformer generation** (e.g. GeoDiff): the molecular *identity is fixed*; the model denoises *coordinates conditioned on the 2D graph*, sampling multiple 3D conformers of one molecule. Useful for docking preparation, shape-based screening, and 3D-QSAR — but it is *not* de novo molecule generation.
- **Pocket-conditioned ligand generation** (e.g. Schneuing et al.): the protein *pocket* supplies the 3D context, and the model generates or denoises a ligand *inside the binding site*, conditioned on pocket atoms, geometry, pharmacophores, or interaction features. It usually needs post-processing to recover valid molecules, followed by docking, filtering, property prediction, and synthesis checks.

5.5 Evaluating Generated Molecules

There is no single score. Basic *diagnostic* checks are **validity** (% syntactically correct), **novelty** (% not in the training set), and **uniqueness** (% distinct among generated). Beyond those:

- **Diversity** — internal diversity (mean pairwise 1−Tanimoto) captures spread but not coverage; *#Circles* counts how many mutually dissimilar molecules fit, capturing chemical-space coverage better.
- **Similarity to a reference set** — compare property distributions via Kullback–Leibler divergence or Kolmogorov–Smirnov distance, or compare in a learned "biologically informed" space via the Fréchet ChemNet Distance.

Fréchet ChemNet Distance (FCD)

Map real and generated molecules to Gaussians using the last hidden layer of a pretrained "ChemNet" (a target-prediction LSTM over SMILES); with means $\boldsymbol{m}, \boldsymbol{m}_w$ and covariances $\boldsymbol{C}, \boldsymbol{C}_w$,

$$\mathrm{FCD}^2 = \|\boldsymbol{m} - \boldsymbol{m}_w\|_2^2 + \mathrm{tr}\Big(\boldsymbol{C} + \boldsymbol{C}_w - 2(\boldsymbol{C}\,\boldsymbol{C}_w)^{1/2}\Big).$$

So FCD compares the **means and covariances of learned latent representations** and measures the distance between the *distributions* of generated vs. real molecules — *not* distances between individual atoms, and *not* computed on raw structures. It conveniently picks up many biases at once (drug-likeness, $\log P$, synthesizability).

6

Chemical Reactions and Synthesis Planning

A generator can dream up a molecule, but a molecule is only useful if someone can actually *make* it. **Synthesis planning** — finding a feasible sequence of reactions from purchasable starting materials to the target — is therefore not a post-processing afterthought but a *core problem* of drug discovery, and it applies equally to human-designed and AI-designed molecules.

6.1 Chemical Reactions

A **chemical reaction** transforms one molecule into another by breaking and forming bonds (electrons move). It is written $a\mathrm{A} + b\mathrm{B} \rightarrow c\mathrm{C} + d\mathrm{D}$ with reactants on the left, products on the right, and (a, b, c, d) the smallest integers that balance the atom counts (conservation of mass). Common types are *combination* ($A + B \rightarrow AB$), *decomposition* ($AB \rightarrow A + B$), *substitution* ($A + BC \rightarrow AC + B$), and *metathesis* ($AB + CD \rightarrow AD + CB$). Which product actually forms is governed by *thermodynamics* (is it energetically favourable?), *kinetics* (is it fast enough?), and *selectivity* (which of several possible products dominates under given conditions) — so planning cares not just about *what* can react but under *which conditions* and with *what yield*.

6.2 ★ Computer-Assisted Synthesis Planning (CASP)

CASP *predates* modern molecular generation: chemists have always needed a route after proposing a target, and symbolic systems date back to Corey & Wipke (1969) and LHASA. Modern ML swapped hand-coded rules for *learned* reaction models, route scoring, and search. Reactions are stored in a SMILES-like language with an **atom-to-atom mapping** (numbering atoms across reactants and products). Datasets include **USPTO** (reactions mined from US patents), CAS / Reaxys / Pistachio, and the Open Reaction Database.

Reaction data is biased

Most reaction datasets contain only *successful* reactions (no failed attempts), records are often *incomplete* (missing conditions, yields), and a recorded route is *not the only ground truth* — other valid routes exist, so benchmark labels underestimate good alternative predictions.

6.3 ★ Predicting Reactions

Two directions, both supervised ML over reaction data:

- **Forward reaction prediction**: input reactants (and sometimes conditions), output the major product.
- **Single-step retrosynthesis** (backward): input a target product, output plausible precursors.

There are two philosophies for how to do it:

Template-free — treat it as *translation*: a sequence-to-sequence "Molecular Transformer" directly maps reactant strings to product strings. Highly accurate (∼90%) but hard for a chemist to inspect (no explicit rules).

Template-based — "neural-symbolic AI": a *neural* part selects a **reaction template**, and a *symbolic* part executes the transformation rule encoded in that template.

A **reaction template** (reaction rule) is a generalised blueprint of a reaction — which substructures must be present and how they transform — designed by experts or extracted from a database. Complex templates don't generalise to new reactants; simple templates apply to too many molecules. Template selection can be framed as *classification* (predict the template with a softmax over K templates) or as *retrieval* from a template database using Modern Hopfield Networks (MHNreact).

AI for chemical reactions — key claims

Some methods rely on **reaction templates** (transformation rules); deep models can be *trained on databases of known reactions*; and **transformer** architectures are used for reaction prediction. What you do *not* need is quantum-mechanical data — ML predicts reactions from reaction databases, not from QM calculations. And synthesis planning is *not* "trivial" nor merely "selecting reactants": it designs a whole *sequence* of reactions and uses *retrosynthetic analysis* to break the target into simpler building blocks.

6.4 Multi-Step Retrosynthesis as a Game

A full synthesis is many steps, so multi-step retrosynthesis repeats single-step decisions until every leaf is a purchasable building block. The elegant framing is as a **game** played with *Monte-Carlo Tree Search + deep reinforcement learning* (the AlphaGo recipe):

The retrosynthesis "game"

- **State**: the current set of molecules still to be made.
- **Action**: selecting reactants (i.e. choosing a reverse reaction / disconnection).
- **Environment**: performs the reaction and returns the new molecules.
- **Reward**: obtained when the current set of molecules are *all* available building blocks — the game is won.

State vs. building blocks — don't swap them

The *state* is the current set of molecules you still need to synthesise; the building blocks are the *goal* you are trying to reach. Saying "the state is the set of available building blocks" is the classic trap — it is false.

Because the search tree is astronomically large (like chess or Go), DNNs predict, for unvisited states, both the *value* (how likely to lead to a solution) and a *policy* (which disconnection to try). In practice ∼50% of molecules can be "solved" with routes that make sense to chemists — though whether the routes actually run, and under which conditions, remains uncertain.

7

Medical Data and Imaging

The second half of the course shifts from molecules to *patients*. The data look different — electronic health records, tabular measurements, medical images — but the same machine-learning discipline applies, with a few twists that are specific to clinical data and that the exam likes to probe.

7.1 Tabular Medical Data

A patient table mixes **data types**, and getting them right drives every preprocessing choice:

- **Categorical (nominal)** — unordered labels, e.g. smoking status (non-/former/current smoker). It is *not* continuous, and you cannot take its mean.
- **Ordinal** — ordered categories without a meaningful numeric distance.
- **Continuous (numeric)** — real-valued measurements like a cholesterol level in mg/dL. Being orderable does *not* make it merely "ordinal".

Imputing missing values

For a skewed *numeric* column (e.g. cholesterol), impute missing values with the **median** (robust to skew), not the mean. For a *categorical* column (e.g. smoking status) you cannot use the mean at all — use the *mode*. Match the imputation to the data type.

7.2 Batch Effects

When samples are processed in different batches (machines, days, labs, reagents), **technical** variation gets baked into the data. *Batch effects* create **artificial differences** that have nothing to do with biology, and if you ignore them a model may learn to **distinguish the batches rather than the actual biological condition**. They are *not* cured by simply collecting more samples, and they are certainly *not* beneficial "variability" — they are a confounder to be corrected (e.g. by batch-correction methods or careful experimental design).

7.3 Domain Shifts

Medical models are deployed on data that drifts away from the training distribution. Using y for the disease status and $\mathbf{x}$ for patient features, the standard taxonomy is:

- **Covariate shift** — the input distribution $p(\mathbf{x})$ changes (so $p(\mathbf{x})$ is certainly *not* immune to shifts).
- **Label / prior shift** — the class prevalence $p(y)$ changes; e.g. during a COVID-19 wave the prevalence of the disease, and thus the probability of observing that phenotype, goes up.
- **Concept shift** — the relationship $p(y \mid \mathbf{x})$ changes; e.g. if the diagnostic procedure that *defines* y is changed (an antigen test replaced by a PCR test), the label's meaning shifts.

Validation accuracy does not buy robustness

High predictive quality on a randomly drawn validation set says *nothing* about whether a model will cope with future domain shifts — it will *not* automatically adapt and maintain quality over time. Monitoring and re-validation are required.

7.4 Medical Imaging

7.4.1 Modalities

Different physics, different images: **X-rays** pass radiation through the body and differentiate tissues by *density* (bone vs. soft tissue); **Computed Tomography (CT)** is X-ray-based (*not* ultrasound), stacking many X-ray projections into a 3D image; **MRI** uses strong magnetic fields and radio waves; *ultrasound* uses sound waves; and *optical coherence tomography* uses *light* (not X-rays).

7.4.2 Tasks and the U-Net

Compared with everyday computer vision, the dominant medical-imaging tasks are **classification** and especially **segmentation** (delineating organs, lesions, tumours pixel by pixel). The signature architecture is the **U-Net**: a *convolutional* network with a **symmetric encoder and decoder** and, crucially, **skip connections** that concatenate encoder feature maps into the corresponding up-sampling level of the decoder, preserving fine spatial detail. It was built for biomedical *segmentation* — *not* object detection, and its encoder is convolutional, *not* self-attention-based.

7.4.3 Pitfalls and Interpretability

The "Clever Hans" effect

A model can appear to perform well while actually relying on *irrelevant features or artifacts* in the data (a ruler in tumour photos, a hospital tag) — shortcut learning. Good test scores can hide it, which is why interpretability matters.

Explainable-AI tools help check *why* a model decided what it did: **Grad-CAM** highlights which image regions drove the prediction; *layer-wise relevance propagation* pushes a relevance signal back through the network layer by layer; *occlusion analysis* masks patches of the **input** image and watches the prediction change (it perturbs inputs, not the training set); and *counterfactual explanations* generate an altered image showing what would have to change for a different diagnosis.

7.5 AlphaFold 2

A landmark where life-science AI met structural biology: **AlphaFold 2** predicts the **three-dimensional structure of a protein from its amino-acid sequence**. It improves accuracy by incorporating **multiple sequence alignments** and *homologous* (template) structures, and its architecture is *attention*-based (the Evoformer) — **not** a convolutional network, and it uses *no* quantum-mechanical calculations.

8

Biological Sequences

The last building block of life-science data is the *sequence*. DNA, RNA and proteins are all linear chains, which makes them a natural fit for the same sequence-modelling toolbox used in NLP.

8.1 What the Sequences Are Made Of

- **DNA** and **RNA** are chains of *nucleic acids* (nucleotides).
- **Proteins** are chains of *amino acids* linked by *peptide bonds*.

These compositions are not interchangeable — RNA is nucleic acid, not lipid — and biological sequences very much *do* follow patterns and rules (codons, motifs, secondary-structure propensities); they are not random strings.

8.2 Deep Learning for Sequences

To feed a sequence to a network it is typically **one-hot encoded** (each nucleotide or amino acid a basis vector). From there, learned **feature extraction** can identify *motifs* — recurring subsequences with biological meaning (binding sites, domains). The full deep-learning toolbox applies to **classification, segmentation, and generative modelling** of sequences, using **1D-convolutions, RNNs, LSTMs, and Transformers** — exactly the architectures that also showed up for SMILES strings in Chapter 5, which is no coincidence: a SMILES string is just another biological-chemical sequence.

Intuition & Mental Model

Notice the unifying thread of the whole course: whether the object is a small molecule (as a graph, fingerprint, or SMILES string), a protein (as a sequence or a 3D structure), an image, or a patient record, the recipe is the same — choose a representation that respects the object's structure and symmetries, then learn f_{θ} from data that is invariably limited, biased and noisy. Everything from QSAR to AlphaFold is a variation on that single theme.